NAME: SANIKA MANAPPA KARANDE

BRANCH: COMPUTER ENGINEERING

DIVISION: F

ROLLNO: 28

PRN: 12414985

BATCH: 1

ASSIGNMENT NO – 6

1] Write a program to keep track of patients as they checked into a medical clinic, assigning patients to doctors on a first-come, first-served basis.

PROGRAM:

```cpp

#include <iostream>
using namespace std;

#define MAX 100

class Queue {
    int front, rear;
    string patients[MAX];

public:
    Queue() {
        front = -1;
        rear = -1;
    }

    bool isFull() {
        return (rear == MAX - 1);
    }

    bool isEmpty() {
        return (front == -1 || front > rear);
    }

```

```
    void enqueue(string name) {
        if (isFull()) {
            cout << "Queue is full. Cannot add more patients.\n";
            return;
        }
        if (front == -1) front = 0; // first patient
        patients[++rear] = name;
        cout << "Patient " << name << " checked in.\n";
    }

    void dequeue() {
        if (isEmpty()) {
            cout << "No patients waiting.\n";
            return;
        }
        cout << "Assigning patient " << patients[front] << " to doctor.\n";
        front++;
    }

    void display() {
        if (isEmpty()) {
            cout << "No patients in queue.\n";
```

```
            if (isEmpty()) {
                cout << "No patients in queue.\n";
                return;
            }
            cout << "Patients waiting: ";
            for (int i = front; i <= rear; i++) {
                cout << patients[i];
                if (i < rear) cout << ", ";
            }
            cout << endl;
        }
};

int main() {
    Queue q;
    int choice;
    string name;

    do {
        cout << "\n--- Medical Clinic System ---\n";
        cout << "1. Check-in patient\n";
        cout << "2. Assign patient to doctor\n";
```

```
        cout << "1. Check-in patient\n";
        cout << "2. Assign patient to doctor\n";
        cout << "3. Show waiting patients\n";
        cout << "4. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
        case 1:
            cout << "Enter patient name: ";
            cin >> name;
            q.enqueue(name);
            break;
        case 2:
            q.dequeue();
            break;
        case 3:
            q.display();
            break;
        case 4:
            cout << "Exiting...\n";
            break;
```


```
            q.display();
            break;
        case 4:
            cout << "Exiting...\n";
            break;
```


```
        default:
            cout << "Invalid choice. Try again.\n";
        }
    } while (choice != 4);

    return 0;
}
```

OUTPUT:

```
--- Medical Clinic System ---
1. Check-in patient
2. Assign patient to doctor
3. Show waiting patients
4. Exit
Enter your choice: 1
Enter patient name: Sanika
Patient Sanika checked in.

--- Medical Clinic System ---
1. Check-in patient
2. Assign patient to doctor
3. Show waiting patients
4. Exit
Enter your choice: 1
Enter patient name: Shital
Patient Shital checked in.
```

```
--- Medical Clinic System ---
1. Check-in patient
2. Assign patient to doctor
3. Show waiting patients
4. Exit
Enter your choice: 1
Enter patient name: Rohit
Patient Rohit checked in.

--- Medical Clinic System ---
1. Check-in patient
2. Assign patient to doctor
3. Show waiting patients
4. Exit
Enter your choice: 2
Assigning patient Sanika to doctor.
```

```
--- Medical Clinic System ---
1. Check-in patient
2. Assign patient to doctor
3. Show waiting patients
4. Exit
Enter your choice: 3
Patients waiting: Shital, Rohit

--- Medical Clinic System ---
1. Check-in patient
2. Assign patient to doctor
3. Show waiting patients
4. Exit
Enter your choice: 4
Exiting...
PS C:\Users\sanik\OneDrive\Documents\DS_ASSI
```

2] Pizza parlour accepting maximum n orders. Orders are served on an FCFS basis. Order once placed can't be cancelled. Write C++ program to simulate the system using circular queue.

PROGRAM:

```cpp

#include <iostream>
using namespace std;

#define MAX 5

class CircularQueue {
    int front, rear;
    int orders[MAX];
    int order_id;
public:
    CircularQueue() {
        front = -1;
        rear = -1;
        order_id = 100;
    }

    bool isFull() {
        return ( (front == 0 && rear == MAX - 1) || (rear + 1) % MAX == front );
    }

    bool isEmpty() {
        return (front == -1);
    }
```

```
    }

    void placeOrder() {
        if (isFull()) {
            cout << "Sorry! No more orders can be placed. Queue is full.\n";
            return;
        }
        order_id++;

        if (front == -1) front = 0;
        rear = (rear + 1) % MAX;
        orders[rear] = order_id;

        cout << "Order placed successfully. Order ID: " << order_id << endl;
    }

    void serveOrder() {
        if (isEmpty()) {
            cout << "No pending orders to serve.\n";
            return;
        }
        cout << "Serving Order ID: " << orders[front] << endl;
```

```
        }
        cout << "Serving Order ID: " << orders[front] << endl;

        if (front == rear) {
            front = -1;
            rear = -1;
        } else {
            front = (front + 1) % MAX;
        }
    }

    void displayOrders() {
        if (isEmpty()) {
            cout << "No pending orders.\n";
            return;
        }

        cout << "Pending Orders: ";
        int i = front;
        while (true) {
            cout << orders[i] << " ";
            if (i == rear) break;
```

```
            if (i == rear) break;
            i = (i + 1) % MAX;
        }
        cout << endl;
    }
};

int main() {
    CircularQueue cq;
    int choice;

    do {
        cout << "\n--- Pizza Parlour System ---\n";
        cout << "1. Place Order\n";
        cout << "2. Serve Order\n";
        cout << "3. Display Pending Orders\n";
        cout << "4. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;

        switch (choice) {
```

```
        switch (choice) {
            case 1:
                cq.placeOrder();
                break;
            case 2:
                cq.serveOrder();
                break;
            case 3:
                cq.displayOrders();
                break;
            case 4:
                cout << "Exiting...\n";
                break;
            default:
                cout << "Invalid choice. Try again.\n";
        }
    } while (choice != 4);

    return 0;
}
```

OUTPUT:

```
--- Pizza Parlour System ---
1. Place Order
2. Serve Order
3. Display Pending Orders
4. Exit
Enter choice: 1
Order placed successfully. Order ID: 101

--- Pizza Parlour System ---
1. Place Order
2. Serve Order
3. Display Pending Orders
4. Exit
Enter choice: 1
Order placed successfully. Order ID: 102
```

```
--- Pizza Parlour System ---
1. Place Order
2. Serve Order
3. Display Pending Orders
4. Exit
Enter choice: 1
Order placed successfully. Order ID: 103

--- Pizza Parlour System ---
1. Place Order
2. Serve Order
3. Display Pending Orders
4. Exit
Enter choice: 2
Serving Order ID: 101
```

```
--- Pizza Parlour System ---
1. Place Order
2. Serve Order
3. Display Pending Orders
4. Exit
Enter choice: 3
Pending Orders: 102 103

--- Pizza Parlour System ---
1. Place Order
2. Serve Order
3. Display Pending Orders
4. Exit
Enter choice: 4
Exiting...
```

3] Write a program that maintains a queue of passengers waiting to see a ticket agent. The program user should be able to insert a new passenger at the rear of the queue, Display the passenger at the front of the Queue, or remove the passenger at the front of the queue. The program will display the number of passengers left in the queue just before it terminates.

PROGRAM:

```

#include <iostream>
using namespace std;

#define MAX 50

class Queue {
    int front, rear, count;
    string passengers[MAX];

public:
    Queue() {
        front = -1;
        rear = -1;
        count = 0;
    }

    bool isFull() {
        return (rear == MAX - 1);
    }

    bool isEmpty() {
        return (front == -1 || front > rear);
    }
```

```
    }

    void enqueue(string name) {
        if (isFull()) {
            cout << "Queue is full! Cannot add more passengers.\n";
            return;
        }
        if (front == -1) front = 0;
        passengers[++rear] = name;
        count++;
        cout << "Passenger " << name << " joined the queue.\n";
    }

    void dequeue() {
        if (isEmpty()) {
            cout << "Queue is empty! No passengers to remove.\n";
            return;
        }
        cout << "Passenger " << passengers[front] << " has been served and removed.\n";
        front++;
        count--;
    }
```

```
    }

    void displayFront() {
        if (isEmpty()) {
            cout << "No passengers waiting.\n";
            return;
        }
        cout << "Passenger at front: " << passengers[front] << endl;
    }

    int getCount() {
        return count;
    }
};

int main() {
    Queue q;
    int choice;
    string name;

    do {
        cout << "\n--- Ticket Agent Queue System ---\n";
```

```
    do {
        cout << "\n--- Ticket Agent Queue System ---\n";
        cout << "1. Add Passenger\n";
        cout << "2. Show Passenger at Front\n";
        cout << "3. Remove Passenger at Front\n";
        cout << "4. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                cout << "Enter passenger name: ";
                cin >> name;
                q.enqueue(name);
                break;
            case 2:
                q.displayFront();
                break;
            case 3:
                q.dequeue();
                break;
            case 4:
                cout << "Program exiting...\n";
```

```
            case 4:
                cout << "Program exiting...\n";
                cout << "Passengers left in queue: " << q.getCount() << endl;
                break;
            default:
                cout << "Invalid choice. Try again.\n";
        }
    } while (choice != 4);

    return 0;
}
```

OUTPUT:

```
--- Ticket Agent Queue System ---
1. Add Passenger
2. Show Passenger at Front
3. Remove Passenger at Front
4. Exit
Enter choice: 1
Enter passenger name: Sanika
Passenger Sanika joined the queue.

--- Ticket Agent Queue System ---
1. Add Passenger
2. Show Passenger at Front
3. Remove Passenger at Front
4. Exit
Enter choice: 1
Enter passenger name: Khushi
Passenger Khushi joined the queue.
```

```
--- Ticket Agent Queue System ---
1. Add Passenger
2. Show Passenger at Front
3. Remove Passenger at Front
4. Exit
Enter choice: 2
Passenger at front: Sanika

--- Ticket Agent Queue System ---
1. Add Passenger
2. Show Passenger at Front
3. Remove Passenger at Front
4. Exit
Enter choice: 3
Passenger Sanika has been served and removed.
```

```
--- Ticket Agent Queue System ---
1. Add Passenger
2. Show Passenger at Front
3. Remove Passenger at Front
4. Exit
Enter choice: 4
Program exiting...
Passengers left in queue: 1
```

4] Write a program to implement multiple queues i.e. two queues using array and perform following operations on it. A. Add Queue, B. Delete from Queue, C. Display Queue.

PROGRAM:

```
#include <iostream>
using namespace std;

#define MAX 10

class Queue {
    int front, rear;
    int arr[MAX];

public:
    Queue() {
        front = -1;
        rear = -1;
    }

    bool isFull() {
        return rear == MAX - 1;
    }

    bool isEmpty() {
        return (front == -1 || front > rear);
    }

    void enqueue(int x) {
```

```
    void enqueue(int x) {
        if (isFull()) {
            cout << "Queue is full! Cannot add.\n";
            return;
        }
        if (front == -1) front = 0;
        arr[++rear] = x;
        cout << "Inserted " << x << " into queue.\n";
    }

    void dequeue() {
        if (isEmpty()) {
            cout << "Queue is empty! Nothing to delete.\n";
            return;
        }
        cout << "Deleted element: " << arr[front] << endl;
        front++;
    }

    void display() {
        if (isEmpty()) {
            cout << "Queue is empty.\n";
            return;
```

```
            cout << "Queue is empty.\n";
            return;
        }
        cout << "Queue elements: ";
        for (int i = front; i <= rear; i++) {
            cout << arr[i] << " ";
        }
        cout << endl;
    }
};

int main() {
    Queue q1, q2;
    int choice, qno, val;

    do {
        cout << "\n--- Multiple Queue System ---\n";
        cout << "1. Add to Queue\n";
        cout << "2. Delete from Queue\n";
        cout << "3. Display Queue\n";
        cout << "4. Exit\n";
```

```
        cout << "4. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                cout << "Enter Queue Number (1 or 2): ";
                cin >> qno;
                cout << "Enter value: ";
                cin >> val;
                if (qno == 1) q1.enqueue(val);
                else if (qno == 2) q2.enqueue(val);
                else cout << "Invalid Queue Number!\n";
                break;

            case 2:
                cout << "Enter Queue Number (1 or 2): ";
                cin >> qno;
                if (qno == 1) q1.dequeue();
                else if (qno == 2) q2.dequeue();
                else cout << "Invalid Queue Number!\n";
```


```
                else cout << "Invalid Queue Number!\n";
```


```
                break;

            case 3:
                cout << "Enter Queue Number (1 or 2): ";
                cin >> qno;
                if (qno == 1) q1.display();
                else if (qno == 2) q2.display();
                else cout << "Invalid Queue Number!\n";
                break;

            case 4:
                cout << "Exiting program...\n";
                break;

            default:
                cout << "Invalid choice! Try again.\n";
        }
    } while (choice != 4);
```

```
        default:
            cout << "Invalid choice! Try again.\n";
        }
    } while (choice != 4);

    return 0;
}
```

OUTPUT:

```
--- Multiple Queue System ---
1. Add to Queue
2. Delete from Queue
3. Display Queue
4. Exit
Enter choice: 1
Enter Queue Number (1 or 2): 1
Enter value: 1
Inserted 1 into queue.

--- Multiple Queue System ---
1. Add to Queue
2. Delete from Queue
3. Display Queue
4. Exit
Enter choice: 1
Enter Queue Number (1 or 2): 1
Enter value: 2
Inserted 2 into queue.
```

```
--- Multiple Queue System ---
1. Add to Queue
2. Delete from Queue
3. Display Queue
4. Exit
Enter choice: 1
Enter Queue Number (1 or 2): 2
Enter value: 3
Inserted 3 into queue.

--- Multiple Queue System ---
1. Add to Queue
2. Delete from Queue
3. Display Queue
4. Exit
Enter choice: 1
Enter Queue Number (1 or 2): 2
Enter value: 4
Inserted 4 into queue.
```

```
--- Multiple Queue System ---
1. Add to Queue
2. Delete from Queue
3. Display Queue
4. Exit
Enter choice: 2
Enter Queue Number (1 or 2): 1
Deleted element: 1

--- Multiple Queue System ---
1. Add to Queue
2. Delete from Queue
3. Display Queue
4. Exit
Enter choice: 3
Enter Queue Number (1 or 2): 2
Queue elements: 3 4
```

```
--- Multiple Queue System ---
1. Add to Queue
2. Delete from Queue
3. Display Queue
4. Exit
Enter choice: 4
Exiting program...
```

5] In a call center, customer calls are handled on a first-come, first-served basis. Implement a queue system using Linked list where:

- **Each customer call is enqueued as it arrives.**
- **Customer service agents dequeue calls to assist customers.**
- **If there are no calls, the system waits.**

PROGRAM:

```
#include <iostream>
using namespace std;

struct Node {
    int callID;
    Node* next;
};

class CallQueue {
    Node* front;
    Node* rear;
    int callCounter;

public:
    CallQueue() {
        front = rear = nullptr;
        callCounter = 100;
    }

    void enqueue() {
        Node* temp = new Node;
        callCounter++;
        temp->callID = callCounter;
        temp->next = nullptr;
```

```
        temp->next = nullptr;

        if (rear == nullptr) {
            front = rear = temp;
        } else {
            rear->next = temp;
            rear = temp;
        }

        cout << "New Call Enqueued. Call ID: " << temp->callID << endl;
    }

    void dequeue() {
        if (front == nullptr) {
            cout << "No calls waiting. Agents are idle.\n";
            return;
        }

        Node* temp = front;
        cout << "Serving Call ID: " << temp->callID << endl;

```

```
            front = front->next;
            if (front == nullptr) rear = nullptr;

            delete temp;
        }

        void display() {
            if (front == nullptr) {
                cout << "No pending calls in queue.\n";
                return;
            }

            cout << "Pending Calls: ";
            Node* temp = front;
            while (temp != nullptr) {
                cout << temp->callID << " ";
                temp = temp->next;
            }
            cout << endl;
        }
    };
```

```
int main() {
    CallQueue cq;
    int choice;

    do {
        cout << "\n--- Call Center Queue System ---\n";
        cout << "1. New Incoming Call (Enqueue)\n";
        cout << "2. Attend Call (Dequeue)\n";
        cout << "3. Show Pending Calls\n";
        cout << "4. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                cq.enqueue();
                break;
            case 2:
                cq.dequeue();
                break;
            case 3:
                cq.display();
                break;
```


```
                cq.display();
                break;
```


```
            case 4:
                cout << "Exiting system...\n";
                break;
            default:
                cout << "Invalid choice. Try again.\n";
        }
    } while (choice != 4);

    return 0;
}
```

OUTPUT:

```
--- Call Center Queue System ---
1. New Incoming Call (Enqueue)
2. Attend Call (Dequeue)
3. Show Pending Calls
4. Exit
Enter choice: 1
New Call Enqueued. Call ID: 101

--- Call Center Queue System ---
1. New Incoming Call (Enqueue)
2. Attend Call (Dequeue)
3. Show Pending Calls
4. Exit
Enter choice: 1
New Call Enqueued. Call ID: 102
```

```
--- Call Center Queue System ---
1. New Incoming Call (Enqueue)
2. Attend Call (Dequeue)
3. Show Pending Calls
4. Exit
Enter choice: 2
Serving Call ID: 101

--- Call Center Queue System ---
1. New Incoming Call (Enqueue)
2. Attend Call (Dequeue)
3. Show Pending Calls
4. Exit
Enter choice: 3
Pending Calls: 102
```

```
--- Call Center Queue System ---
1. New Incoming Call (Enqueue)
2. Attend Call (Dequeue)
3. Show Pending Calls
4. Exit
Enter choice: 4
Exiting system...
```